

ATIVIDADE - LISTA ENCADEADA (ESD)

Disciplina: Estrutura de Dados 1

Enunciado resolvido

1. Struct do nó com inteiro e ponteiro para próximo nó.
2. Função para inicializar lista vazia.
3. Função para inserir no início da lista.
4. Uso solicitado: **Lista* l; l = inicializa(); insere(l, 10); insere(l, 20);**
5. Função para imprimir todos os elementos.
6. Função que retorna 1 se vazia e 0 caso contrário.
7. Função que retorna quantidade de elementos.
8. Função que retorna a soma dos valores.
9. Função que retorna ponteiro do nó buscado ou NULL.
10. Função que retorna o maior valor da lista.

Arquivo: atividade_lista_encadeada.c

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct no {
    int valor;
    struct no *prox;
} No;

typedef struct lista {
    No *inicio;
} Lista;

Lista *inicializa(void) {
    Lista *l = (Lista *)malloc(sizeof(Lista));
    if (l == NULL) {
        return NULL;
    }
    l->inicio = NULL;
    return l;
}

int insere(Lista *l, int valor) {
    No *novo;
    if (l == NULL) {
        return 0;
    }
```

```
}
novo = (No *)malloc(sizeof(No));
if (novo == NULL) {
    return 0;
}
novo->valor = valor;
novo->prox = l->inicio;
l->inicio = novo;
return 1;
}

void imprime(const Lista *l) {
    No *p;
    if (l == NULL) {
        printf("Lista invalida.\n");
        return;
    }
    p = l->inicio;
    printf("Lista: ");
    while (p != NULL) {
        printf("%d ", p->valor);
        p = p->prox;
    }
    printf("\n");
}

int vazia(const Lista *l) {
    return (l == NULL || l->inicio == NULL) ? 1 : 0;
}

int quantidade(const Lista *l) {
    int qtd = 0;
    No *p;
    if (l == NULL) {
        return 0;
    }
    p = l->inicio;
    while (p != NULL) {
        qtd++;
        p = p->prox;
    }
    return qtd;
}

int soma(const Lista *l) {
    int total = 0;
    No *p;
    if (l == NULL) {
        return 0;
    }
    p = l->inicio;
```

```
    while (p != NULL) {
        total += p->valor;
        p = p->prox;
    }
    return total;
}

No *busca(const Lista *l, int valor) {
    No *p;
    if (l == NULL) {
        return NULL;
    }
    p = l->inicio;
    while (p != NULL) {
        if (p->valor == valor) {
            return p;
        }
        p = p->prox;
    }
    return NULL;
}

int maior(const Lista *l) {
    int maiorValor;
    No *p;
    if (vazia(l)) {
        return INT_MIN;
    }
    p = l->inicio;
    maiorValor = p->valor;
    p = p->prox;
    while (p != NULL) {
        if (p->valor > maiorValor) {
            maiorValor = p->valor;
        }
        p = p->prox;
    }
    return maiorValor;
}
```